Quantum Krylov subspace methods

A short note for numerical linear algebraists

Martin Stoll John Pearson

November 2026

Abstract

Quantum Krylov methods compute the lowest eigenvalue of a huge Hermitian matrix by running a Rayleigh–Ritz procedure. The quantum computer supplies the entries of the small projected matrices and a classical computer solves the resulting generalized eigenvalue problem. This note explains the idea in standard linear algebra notation. It covers what a quantum device can and cannot do, which Krylov bases it can generate, and why the main numerical difficulty is an ill-conditioned Gram matrix whose entries carry statistical noise. A small experiment illustrates the points, and we list open questions where numerical linear algebra can contribute.

1 Motivation

Many problems in quantum physics and chemistry reduce to computing the smallest eigenvalue λ_0 of a Hermitian matrix $H \in \mathbb{C}^{n\times n}$. Physicists call H the *Hamiltonian*, λ_0 the *ground state energy* and its eigenvector u_0 the *ground state*. For a system of N particles that each have two possible configurations (spins, qubits, spin orbitals), H acts on $\mathbb{C}^2 \otimes \cdots \otimes \mathbb{C}^2$, so $n = 2^N$. For $N = 50$, a single vector in double precision already takes 16 petabytes. Lanczos, Davidson and their relatives are the methods of choice for moderate N. Beyond about $N \approx 50$, any method that stores vectors explicitly is hopeless, and one turns to compressed formats such as tensor networks.

A quantum computer with N qubits stores a vector of length 2^N *implicitly* and can multiply it by certain unitary matrices cheaply. The catch is that the vector can never be read out. One only gets random samples whose statistics encode quadratic forms. Quantum Krylov methods (also called quantum subspace diagonalization) are a pragmatic way of combining this ability with the Rayleigh–Ritz principle [1–7]. Experiments on current hardware now reach subspace dimensions and problem sizes of practical interest [8]. For an NLA reader these methods are attractive because their core difficulties are classical: a non-orthogonal Krylov basis, an ill-conditioned Gram matrix, and a generalized eigenvalue problem with noisy data.

2 A quantum computer as a restricted linear algebra device

This note assumes no background beyond linear algebra. This section translates the quantum computing vocabulary into statements about vectors and matrices. Throughout, N is the number of qubits and $n = 2^N$.

We write $\mathrm{poly}(N)$ for any quantity bounded by cN^p with constants c, p that do not depend on N, such as $2N - 1$ or $5N^3$. Since $N = \log_2 n$, this is the same as polylogarithmic in the matrix size n, i.e. $O(\log^p n)$, and therefore tiny compared with n. By contrast, $2^N = n$ is *not* $\mathrm{poly}(N)$. "Efficient" or "cheap" in this note always means $\mathrm{poly}(N)$. With several arguments, e.g. $\mathrm{poly}(N, t, 1/\varepsilon)$, the bound is a polynomial in all of them.

2.1 Vectors

The memory of a quantum computer with N qubits holds a single unit vector $v \in \mathbb{C}^n$, called the *state*. One qubit holds a unit vector in $\mathbb{C}^2$, and N qubits hold a vector in $\mathbb{C}^2 \otimes \cdots \otimes \mathbb{C}^2 = \mathbb{C}^n$. The standard basis vectors are indexed by bit strings,

$$e_{b_1 b_2 \ldots b_N} = e_{b_1} \otimes e_{b_2} \otimes \cdots \otimes e_{b_N}, \qquad e_0 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \; e_1 = \begin{bmatrix} 0 \\ 1 \end{bmatrix},$$

where each $b_i \in \{0, 1\}$ is a binary digit (a *bit*). The bit string $b = b_1 b_2 \ldots b_N$ is the binary representation of the position of the entry, counted from 0. For example, for $N = 2$ the vectors $e_{00}, e_{01}, e_{10}, e_{11}$ are the first to fourth standard basis vectors of $\mathbb{C}^4$. We identify b with the integer $0, \ldots, n-1$ it represents, so v_b is an entry of v. A *product state* is a Kronecker product $a_1 \otimes \cdots \otimes a_N$ of unit vectors $a_i \in \mathbb{C}^2$. It is described by $2N$ numbers, whereas a general v needs $n = 2^N$. Physicists write $|v\rangle$ for v and $\langle w|v\rangle$ for $w^* v$. We do not use this notation.

A vector that is *not* a product state is called *entangled*. In linear algebra terms entanglement is a rank property. For $N = 2$, reshape $v \in \mathbb{C}^4$ into the 2×2 matrix $V = \left[\begin{smallmatrix} v_{00} & v_{01} \\ v_{10} & v_{11} \end{smallmatrix}\right]$. Then $v = a_1 \otimes a_2$ holds exactly when $V = a_1 a_2^T$, i.e. when $\operatorname{rank} V = 1$. For example, $(e_{00} + e_{11})/\sqrt{2}$ gives $V = I/\sqrt{2}$ of rank 2, so it is entangled. For general N, split the qubits into a first group of k and a second group of $N - k$, and reshape v into a $2^k \times 2^{N-k}$ matrix. Its rank (the *Schmidt rank*) and its singular values (the *Schmidt coefficients*) measure the entanglement between the two groups. A product state has rank one for every such split. Vectors with small ranks for the splits $k = 1, \ldots, N-1$ are exactly those with a cheap tensor-train representation (physicists call it a *matrix product state*), which is why tensor networks work well classically for weakly entangled problems. Ground states of many one-dimensional problems are of this kind, which is why classical methods such as DMRG succeed there. A generic vector, however, has ranks growing exponentially in N. Many ground states in two and three dimensions and in chemistry also have large ranks, as do the time-evolved vectors $e^{-\mathrm{i}tH} v_0$ used below, whose ranks typically grow quickly with t. Storing and manipulating such strongly entangled vectors without writing them down is what a quantum computer offers.

The device always starts from $e_{0\ldots0}$, and all further vectors are obtained by multiplying with unitary matrices. A start vector v_0 is therefore available only if $v_0 = Q_0 e_{0\ldots0}$ for a unitary Q_0 that can be applied cheaply in the sense of Section 2.4.

2.2 Reading out: measurement

What can be read out of the device shapes everything that follows, so we discuss it before the computations. On a classical computer the entries of v sit in memory and can be inspected. On a quantum computer they cannot. The only way to extract information is a *measurement*, which has three properties a linear algebraist should keep in mind.

1. A measurement returns a random index, not the vector. Measuring the state v returns one bit string b, chosen at random with probability

$$p_b = |v_b|^2.$$

This explains why states are unit vectors: the numbers $|v_b|^2$ are nonnegative and sum to $\|v\|^2 = 1$, so they form a probability distribution on $\{0, \ldots, n-1\}$. The vector v is the device's internal description of this distribution, and we only ever see samples from it. The device is best thought of as a random number generator whose distribution we shape by linear algebra (the choice of unitaries) but whose parameters we cannot read.

2. A measurement destroys the vector. After the measurement the state is e_b, and v is gone. A second sample requires preparing v again from $e_{0\ldots0}$, i.e. running the whole circuit again. One such run is called a *shot*. A probability p is therefore estimated as the fraction of N_s shots with the desired outcome. The standard deviation is $\sqrt{p(1-p)/N_s} \le 1/(2\sqrt{N_s})$, so accuracy 10^{-3} needs on the order of 10^5–10^6 shots. Double precision is out of reach. Individual entries are hopeless as well: for a typical v each p_b is about $1/n = 2^{-N}$, so one would need about 2^N shots to see a particular b even once. Useful outputs are therefore a few aggregate numbers, such as the probability of a large set of outcomes.

3. Phases are invisible. v and $e^{\mathrm{i}\varphi}v$, or more generally v and Dv with D a unitary diagonal matrix, give the same probabilities, because only the moduli $|v_b|$ enter. The information we want, however, is phase information: eigenvalues appear as phases $e^{-\mathrm{i}\lambda t}$, and we need inner products v^*Wv. The remedy is *interference*. Add two vectors so that their relative phase changes the norm of the sum, e.g. $\|v + Wv\|^2 = 2 + 2\,\mathrm{Re}(v^*Wv)$ for unitary W. Sections 2.4 and 2.5 show how the device does this.

Measuring some of the qubits. Often only a few qubits are measured, typically extra qubits added for the purpose. Measuring the first qubit splits the vector into its two halves $x = [x_0; x_1]$ (here and below $[x; y]$ denotes the vector with block x on top of block y), where x_0 holds the entries whose first bit is 0 and x_1 those whose first bit is 1. The outcome 0 has probability $\|x_0\|^2$, and afterwards the state is $[x_0; 0]/\|x_0\|$. The outcome 1 has probability $\|x_1\|^2$, and the state becomes $[0; x_1]/\|x_1\|$. In linear algebra terms, the measurement applies one of the two orthogonal projectors $\mathrm{diag}(I, 0)$ and $\mathrm{diag}(0, I)$, chosen at random with probability equal to the squared norm of the projected vector, and then normalizes. This is the only non-unitary operation available. It is the mechanism behind the two constructions used throughout the note:

- to *estimate a number*, arrange by unitaries that it equals the squared norm of a block, then measure (the Hadamard test, Section 2.5);
- to *apply a non-unitary matrix* A, arrange that Av appears in a block, then measure and keep the run only if this block was selected (block encodings, Section 2.4).

2.3 The matrix

The $n \times n$ matrix H is never stored. It is given as a sum of $L = \mathrm{poly}(N)$ *Pauli strings* (for the Ising chain below, $L = 2N - 1$),

$$H = \sum_{\ell=1}^{L} \alpha_\ell P_\ell, \qquad P_\ell = \sigma_{\ell,1} \otimes \cdots \otimes \sigma_{\ell,N}, \tag{1}$$

where $\alpha_\ell \in \mathbb{R}$. Each term P_ℓ is a Kronecker product of N matrices of size 2×2, one per qubit. The factor $\sigma_{\ell,i}$ is the one in position i of the ℓ-th term, and each factor is one of the four matrices

$$I, \quad X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad Y = \begin{bmatrix} 0 & -\mathrm{i} \\ \mathrm{i} & 0 \end{bmatrix}, \quad Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

(the *Pauli matrices*). For example, with $N = 4$ the string $P = Z \otimes Z \otimes I \otimes I$ has $\sigma_1 = \sigma_2 = Z$ and $\sigma_3 = \sigma_4 = I$. The 4^N Pauli strings form an orthogonal basis of $\mathbb{C}^{n\times n}$ with respect to $\langle A, B\rangle = \mathrm{trace}(A^*B)$, so every matrix can be written in the form (1). The point is that physically relevant H need only $L = \mathrm{poly}(N)$ terms, because most factors are I. Each term is stored as its coefficient α_ℓ and a word of length N over the alphabet $\{I, X, Y, Z\}$, so storing H needs only

$O(LN)$ numbers. Each P_ℓ is Hermitian and unitary. A standard example, used below, is the transverse-field Ising chain (a simple model of interacting magnetic moments)

$$H = -\sum_{i=1}^{N-1} Z_i Z_{i+1} - h\sum_{i=1}^{N} X_i, \qquad Z_i = I \otimes \cdots \otimes I \otimes Z \otimes I \otimes \cdots \otimes I \ (Z \text{ in position } i),$$

and X_i is defined in the same way. H is not unitary, so it cannot be applied directly. Section 2.4 explains what can be applied instead.

2.4 Which unitary matrices can be applied

Gates and circuits. The hardware applies a small set of elementary unitaries, the *gates*. A one-qubit gate is

$$I_{2^{i-1}} \otimes u \otimes I_{2^{N-i}}, \qquad u \in \mathbb{C}^{2\times 2} \text{ unitary},$$

i.e. it acts on the i-th Kronecker factor only. A two-qubit gate acts on two neighbouring factors $i, i+1$,

$$I_{2^{i-1}} \otimes w \otimes I_{2^{N-i-1}}, \qquad w \in \mathbb{C}^{4\times 4} \text{ unitary}.$$

Applying one gate takes a fixed physical time, independent of n. The device thus performs a "matvec" with a $2^N \times 2^N$ matrix in one step, but only for these very special matrices. A *circuit* is a product

$$Q = \Gamma_q \cdots \Gamma_2 \Gamma_1,$$

where each $\Gamma_j \in \mathbb{C}^{n\times n}$ is a one- or two-qubit gate of the form above, with its own position i and its own small unitary u or w. Γ_1 is applied first. The cost of the circuit is the number q of gates. Gates acting on disjoint factors commute; for example $(u\otimes I)(I\otimes u') = (I\otimes u')(u\otimes I) = u\otimes u'$. Such gates can be applied simultaneously. The number of such parallel layers is the *depth*, which determines the run time.

An important construction is the *controlled* version of a unitary $U \in \mathbb{C}^{n\times n}$,

$$C(U) = \begin{bmatrix} I_n & 0 \\ 0 & U \end{bmatrix} = e_0 e_0^T \otimes I_n + e_1 e_1^T \otimes U \in \mathbb{C}^{2n\times 2n}. \tag{2}$$

It uses one extra qubit (the first Kronecker factor) and applies U to the lower half of the vector only: $C(U)[x;y] = [x;Uy]$. If U has a circuit, $C(U)$ has one with a constant factor more gates. The simplest example is the permutation matrix $C(X) \in \mathbb{C}^{4\times 4}$ (the CNOT gate).

Why controlled unitaries matter. Applying U to v and measuring reveals only the numbers $|(Uv)_b|^2$ (Section 2.2, point 3). Inner products such as v^*Uv, and in particular phases, cannot be seen this way. If v is an eigenvector, $Uv = e^{-\mathrm{i}\varphi}v$, then measuring Uv gives exactly the same probabilities as measuring v. The phase φ, which carries the eigenvalue we are after, is invisible. A phase only becomes visible *relative* to a vector that did not receive it, and $C(U)$ produces such a pair inside one vector. With the 2×2 unitary (the *Hadamard gate*)

$$h = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \qquad \text{one gets} \qquad (h\otimes I_n)\, C(U)\, (h\otimes I_n) = \frac{1}{2}\begin{bmatrix} I+U & I-U \\ I-U & I+U \end{bmatrix}. \tag{3}$$

This product of three cheap unitaries contains $\frac{1}{2}(I+U)$ as its leading block. Applied to $[v;0]$, the top block of the result is $\frac{1}{2}(v+Uv)$. Its squared norm $\frac{1}{2}\big(1+\mathrm{Re}(v^*Uv)\big)$ is the probability of measuring 0 on the extra qubit. In this way $C(U)$ turns an inner product into a measurable probability. This is the Hadamard test of Section 2.5, and the basis of all eigenvalue algorithms in this note. For an eigenvector the probability is $\frac{1}{2}(1+\cos\varphi)$, which reveals φ; this is also the starting point of phase estimation.

unitary matrix	remark	cost (gates)
$u_1 \otimes \cdots \otimes u_N$, $u_i \in \mathbb{C}^{2\times 2}$	one gate per factor; includes all Pauli strings P_ℓ	N, depth 1
$\operatorname{diag}\left(e^{\mathrm{i}\varphi(b)}\right)_b$	φ computable by a short classical (Boolean) circuit	poly(N)
permutation $e_b \mapsto e_{\pi(b)}$	π computable by a short classical circuit, e.g. $b \mapsto b+1 \bmod n$	poly(N)
$e^{-\mathrm{i}\theta P}$, P a Pauli string	$P = QDQ^*$ with Q a Kronecker product and D diagonal (see text)	$O(N)$
unitary DFT matrix F_n	$(F_n)_{jk} = \omega^{jk}/\sqrt{n}$, $\omega = e^{2\pi\mathrm{i}/n}$ (quantum Fourier transform)	$O(N^2) = O(\log^2 n)$
$C(U)$	block diagonal, see (2)	const × cost(U)
$e^{-\mathrm{i}tH}$, H as in (1)	approximately, by products of $e^{-\mathrm{i}\theta P_\ell}$ (see text)	poly($N, t, 1/$accuracy)
U^k	no shortcut in general	$k\times$ cost(U)
block encoding of A	non-unitary A, applied with probability < 1 (see text)	depends on A

Table 1: Unitary matrices of size $n = 2^N$ that a quantum computer can apply cheaply.

Controlled unitaries play a second role as well. A circuit cannot branch on the entries of v, so $C(U)$ is the linear-algebra replacement of an if-statement: apply U to the part of the vector where the extra qubit is 1. With several extra qubits, $\sum_\ell e_\ell e_\ell^T \otimes P_\ell$ applies P_ℓ to the ℓ-th block. Combined with (3)-type mixing of the blocks, this is how linear combinations $\sum_\ell \alpha_\ell P_\ell$ such as H are block encoded (see below).

What is cheap. Every unitary in $\mathbb{C}^{n\times n}$ is a product of gates. However, a generic unitary needs on the order of $n^2 = 4^N$ gates, because the unitary matrices form a set of real dimension n^2 and each gate contributes at most 16 real parameters. "Can be applied" therefore means in practice: a circuit with polynomially many gates in N, i.e. polylogarithmically many in n. These matrices are extremely structured. They resemble classical fast transforms, but the gap is far larger: the FFT costs $O(n \log n)$ flops, whereas the corresponding quantum circuit needs only $O(\log^2 n)$ gates. Table 1 lists the families relevant for this note.

Exponentials of Pauli strings. Each 2×2 Pauli matrix is diagonalized by a fixed 2×2 unitary: $X = hZh^*$ and $Y = (sh)Z(sh)^*$ with h from (3) and $s = \operatorname{diag}(1, \mathrm{i})$. Hence every Pauli string satisfies $P = QDQ^*$, where Q is a Kronecker product of 2×2 unitaries and D is a Kronecker product of I's and Z's. D is diagonal with entries ± 1, namely the parity of the bits of b in the positions where D has a Z. Therefore $e^{-\mathrm{i}\theta P} = Qe^{-\mathrm{i}\theta D}Q^*$, and $e^{-\mathrm{i}\theta D}$ is a diagonal matrix of the type in the second row of Table 1.

Time evolution. Each single term $e^{-\mathrm{i}t\alpha_\ell P_\ell}$ is cheap, but the matrices P_ℓ do not commute, so $e^{-\mathrm{i}tH} \neq \prod_\ell e^{-\mathrm{i}t\alpha_\ell P_\ell}$. For two Hermitian matrices A, B and a step $\delta > 0$ one has

$$\left\| e^{-\mathrm{i}\delta A} e^{-\mathrm{i}\delta B} - e^{-\mathrm{i}\delta(A+B)} \right\|_2 \leq \tfrac{\delta^2}{2} \left\| AB - BA \right\|_2,$$

so splitting the exponential is accurate only for *short* times. The remedy is to cut t into r steps of length $\delta = t/r$. Since $e^{-\mathrm{i}tH} = \left(e^{-\mathrm{i}(t/r)H}\right)^r$ exactly, one only has to split each short step:

$$e^{-\mathrm{i}tH} \approx \Big(\prod_{\ell=1}^{L} e^{-\mathrm{i}(t/r)\alpha_\ell P_\ell} \Big)^r \qquad \text{(Lie–Trotter)}.$$

Each of the r factors has error $O\big((t/r)^2\big)$. For unitaries, errors add up at most linearly: $\|U_1^r - U_2^r\|_2 \le r\,\|U_1 - U_2\|_2$ (telescoping sum). The total error is therefore $O(t^2/r)$, with a constant involving the commutators of the P_ℓ, and it can be made small by increasing r. The circuit consists of Lr Pauli exponentials, so accuracy ε costs $r = O(t^2/\varepsilon)$ steps. Higher-order product formulas and other methods reduce the cost to roughly linear in t. For a general H the cost of $e^{-\mathrm{i}tH}$ *must* grow linearly with t (this is the "no fast-forwarding" theorem). Classically, H^k or U^k are free once H is diagonalized; here U^k costs k times as much as U. In a Krylov method, the last basis vector is therefore the most expensive one.

Block encodings: using non-unitary matrices. A non-unitary $A \in \mathbb{C}^{n\times n}$ with $\|A\|_2 \le \alpha$ can be used through a larger unitary

$$W = \begin{bmatrix} A/\alpha & * \\ * & * \end{bmatrix} \in \mathbb{C}^{2^a n \times 2^a n},$$

which needs a extra qubits. The blocks $*$ are whatever makes W unitary. A unitary completion always exists, e.g. for $a = 1$ and $B = A/\alpha$ (so $\|B\|_2 \le 1$)

$$W = \begin{bmatrix} B & (I - BB^*)^{1/2} \\ (I - B^*B)^{1/2} & -B^* \end{bmatrix}.$$

So in principle A can be any matrix. In practice, what matters is whether W has a *short* circuit, and for a generic A it does not. The completion above involves the dense matrix $(I - BB^*)^{1/2}$. A generic A has n^2 independent entries, and just feeding them into the device already costs at least n^2 operations, which is more than a classical matvec. Exactly as for unitaries, only structured A can be used cheaply.

Example: block encoding of H. For $H = \sum_{\ell=1}^{L} \alpha_\ell P_\ell$ as in (1), take $a = \lceil \log_2 L \rceil$ extra qubits, $\alpha = \sum_\ell |\alpha_\ell|$, and the unit vector $r = \big(\sqrt{|\alpha_\ell|/\alpha}\big)_{\ell=1}^{L} \in \mathbb{C}^{2^a}$ (padded with zeros). Let R be a unitary on $\mathbb{C}^{2^a}$ whose first column is r, and let $S = \sum_\ell e_\ell e_\ell^T \otimes \mathrm{sign}(\alpha_\ell) P_\ell$, with e_ℓ the ℓ-th standard basis vector of $\mathbb{C}^{2^a}$, be the block diagonal matrix that applies $\pm P_\ell$ to the ℓ-th block, a controlled unitary in the sense of (2). Then the matrix $W = (R^* \otimes I_n)\, S\, (R \otimes I_n)$ has the leading $n \times n$ block

$$(r^* \otimes I_n)\, S\, (r \otimes I_n) = \sum_\ell \frac{|\alpha_\ell|}{\alpha} \mathrm{sign}(\alpha_\ell) P_\ell = \frac{H}{\alpha}.$$

R acts on only $a \approx \log_2 L$ qubits and S consists of L controlled Pauli strings, so W costs $\mathrm{poly}(N)$ gates. This is called the *linear combination of unitaries*.

Which A are cheap? The main families are: (i) linear combinations $\sum_\ell \alpha_\ell U_\ell$ of unitaries with short circuits, as just shown, with $\alpha = \sum_\ell |\alpha_\ell|$; (ii) sparse matrices with at most s nonzeros per row and column whose positions and values can be computed from the row index by a short classical circuit, with $\alpha = s \max_{ij} |a_{ij}|$; and (iii) everything obtained from these by products, linear combinations, Kronecker products and polynomials (below). A need not be Hermitian or even square; in this note, however, $A = H$ throughout.

The normalization α is not a detail. It may be much larger than $\|A\|_2$; for example, $\sum_\ell |\alpha_\ell|$ often exceeds $\|H\|_2$ considerably. A large α costs twice. First, the success probability $\|Av\|^2/\alpha^2$ below becomes small. Second, the eigenvalue gap of A/α shrinks to $(\lambda_1 - \lambda_0)/\alpha$, so polynomial filters in A/α need a higher degree.

Applying W to $[v; 0]$ gives $[Av/\alpha;\, w]$. Measuring the extra qubits (Section 2.2) yields the top block with probability $\|Av\|^2/\alpha^2$, and the state is then $Av/\|Av\|$. Otherwise the run is discarded. So A is applied only with some probability, and the result is always normalized. Products of W, W^* and simple diagonal unitaries give block encodings

$$\begin{bmatrix} p(A/\alpha) & * \\ * & * \end{bmatrix}$$

of polynomials p of degree d with $|p(x)| \le 1$ on $[-1,1]$ (for Hermitian A), at the cost of about d applications of W [9]. For the Chebyshev polynomials T_k this is particularly simple.

Classical analogue. The idea is the same as embedding a Toeplitz matrix $T \in \mathbb{C}^{n\times n}$ into a circulant $C \in \mathbb{C}^{2n\times 2n}$ with leading block T for fast matvecs. Pad with zeros, apply the larger and better structured matrix, and keep the top block: $C[x;0] = [Tx;*]$ just as $W[v;0] = [Av/\alpha;*]$. In both cases the free blocks $*$ are chosen to make the large matrix cheap to apply, by the FFT in one case and by a short circuit in the other. There are three differences. (i) Classically one simply discards the bottom block. Here discarding is a projection, which is not unitary; it is done by measuring the extra qubits, succeeds only with probability $\|Av\|^2/\alpha^2$, and returns the normalized vector. (ii) The scaling by α is forced by unitarity ($\|W\|_2 = 1$), whereas a circulant embedding needs no scaling. (iii) Classically $T^k x$ is obtained by embedding and truncating k times. Here powers do not compress: the leading block of W^2 is B^2 plus a contribution from the $*$ blocks, not B^2. Truncating by measurement after every step multiplies the success probabilities, which quickly becomes hopeless. Polynomial methods [9] therefore interleave W with simple reflections that act as a "coherent" substitute for the truncation. In operator theory this corresponds to the difference between a unitary dilation of B and a dilation of all powers of B (Sz.-Nagy, Egerváry).

What is not possible.

- Applying a generic dense unitary, which needs exponentially many gates.
- Applying H, or any non-unitary matrix, deterministically. Block encodings apply it only with some probability, and the result is normalized.
- Nonlinear operations on the vector: normalization, Gram–Schmidt, dividing by an inner product, or any step whose choice depends on the entries of v (other than through measurement).
- Copying: no unitary Q satisfies $Q(v \otimes e_{0\ldots0}) = v \otimes v$ for all v, since $v \mapsto v \otimes v$ is not linear (the "no-cloning" theorem).
- Reading out v. Sections 2.2 and 2.5 describe what can be extracted.

Hardware today. Each two-qubit gate is faulty with probability around 10^{-3}, so circuits with more than about 10^3–10^4 two-qubit gates produce mostly noise. Moreover, gates usually act only on physically neighbouring qubits. Future *error-corrected* hardware will store each qubit redundantly in many physical qubits and correct errors on the fly, which allows much longer circuits at a large overhead in qubits.

2.5 Output: estimating inner products

We now combine the pieces. By Section 2.2, a number can be read out only if it is the probability of a measurement outcome, i.e. the squared norm of a block, and phases must first be converted into norms by interference.

The Hadamard test estimates v^*Wv for a unitary W with a circuit. It uses one extra qubit, so all vectors lie in $\mathbb{C}^{2n}$, and h is the Hadamard gate from (3). Steps 1–3 apply exactly the product in (3).

1. Prepare $[v;0]$ and apply $h \otimes I_n$: $\quad \frac{1}{\sqrt{2}}[v;\, v]$.
2. Apply $C(W)$ from (2): $\quad \frac{1}{\sqrt{2}}[v;\, Wv]$.

	classical Lanczos	quantum Krylov
storage of v	explicit, $O(n)$	implicit, $N = \log_2 n$ qubits
basis generator	$v \mapsto Hv$	$v \mapsto f(H)v$, $f(H)$ unitary or block encoded
inner products	exact up to rounding	sampled, error $\sim N_s^{-1/2}$
orthogonalization	Gram–Schmidt (Lanczos recurrence)	only via G (CholeskyQR-like)
projected problem	tridiagonal T_m	dense pencil (M, G), G ill-conditioned

Table 2: Lanczos versus quantum Krylov methods.

3. Apply $h \otimes I_n$: $\frac{1}{2}[v + Wv;\, v - Wv]$.

4. Measure the extra qubit. The outcome is 0 with probability $\frac{1}{4}\|v + Wv\|^2 = \frac{1}{2}\big(1 + \mathrm{Re}(v^*Wv)\big)$.

With $s = +1$ for outcome 0 and $s = -1$ for outcome 1, we get $\mathbb{E}[s] = \mathrm{Re}(v^*Wv)$. Starting step 2 instead from $\frac{1}{\sqrt{2}}[v;\, -\mathrm{i}v]$ gives $\mathrm{Im}(v^*Wv)$. Averaging N_s runs (*shots*) gives an estimate with standard deviation at most $N_s^{-1/2}$, so an accuracy of 10^{-3} costs about 10^6 runs. Hardware errors add further, partly systematic, perturbations. All quantities needed by the Krylov methods below have the form $v_0^*Wv_0$, with $W = U^k$ or $W = P_\ell U^k$.

Table 2 summarizes the consequences for a Krylov method. The two key restrictions are that inner products are noisy and that the vectors *cannot be orthogonalized* on the device, since Gram–Schmidt needs the vectors themselves (Section 3).

3 Rayleigh–Ritz with a non-orthogonal basis

Let $v_0, \ldots, v_{m-1}$ be vectors that can be prepared on the device and let $V = [v_0, \ldots, v_{m-1}] \in \mathbb{C}^{n\times m}$. V is never formed. We only need the $m \times m$ matrices

$$G = V^*V, \qquad M = V^*HV, \qquad G_{jk} = v_j^*v_k, \quad M_{jk} = v_j^*Hv_k, \tag{4}$$

which the quantum computer estimates entry by entry. A classical computer then solves the generalized Hermitian eigenvalue problem

$$Mc = \theta\, Gc, \tag{5}$$

and the smallest Ritz value $\theta_{\min}$ approximates λ_0, with Ritz vector Vc. In exact arithmetic, and with G positive definite, this is standard Rayleigh–Ritz on range(V). By Courant–Fischer, $\theta_{\min} \geq \lambda_0$, and $\theta_{\min}$ decreases monotonically as the subspace grows. Classically one would orthonormalize V first. Here this is not possible on the vectors themselves (see below), so (5) is the generalized eigenvalue problem in the original, typically very ill-conditioned, basis. Everything that follows is about which vectors v_k to choose and how to cope with G.

Why not orthogonalize? Classical Gram–Schmidt works on the vectors,

$$w = v_k - \sum_{j<k} (q_j^*v_k)\, q_j, \qquad q_k = w/\|w\|,$$

and each ingredient fails on the device.

- The coefficients $q_j^*v_k$ are not available inside the device. They can only be estimated by repeated measurement (Section 2.5), i.e. read out as noisy numbers.
- The map $v_k \mapsto q_k$ depends on v_k itself (through the coefficients and the normalization), so it is not multiplication by a fixed unitary matrix, which is all the device can do (Section 2.4).

- The memory holds one vector, which cannot be copied. So $q_0, \ldots, q_{k-1}$ cannot be kept alongside v_k to subtract them.

What remains is to orthogonalize *in coefficient space*. Every orthonormal basis of range(V) has the form VR^{-1} with an invertible R, and choosing the Cholesky factor $G = R^*R$ turns (5) into the standard Hermitian problem $R^{-*}MR^{-1}y = \theta y$. This is exactly the classical CholeskyQR algorithm: orthogonalization using only the Gram matrix. CholeskyQR is known to be unstable because $\kappa_2(G) = \kappa_2(V)^2$. Once $\kappa_2(V)^2$ exceeds the inverse accuracy of G, the factor R is meaningless. Classically that accuracy is the unit roundoff $\mathbf{u}$, but here it is the noise level $\varepsilon \approx 10^{-3}$. Modified Gram–Schmidt and Householder avoid the squaring by working on the vectors, which is precisely what the device cannot do. This is the root of the numerical difficulties in Section 5.

One can also aim for orthonormal vectors directly. The Lanczos vectors are $q_k = \pi_k(H)v_0$ for polynomials π_k whose coefficients follow from the moments of Section 4.4, and with a block encoding $\pi_k(H)$ can be applied (Section 2.4). However, the coefficients of π_k must first be computed classically from noisy measured data, which is the same ill-conditioned problem in another form.

The Ritz vector itself is not available, only its coefficient vector c. Other quantities, such as expectation values x^*Ax of observables A in the Ritz vector $x = Vc$, require further matrices V^*AV estimated in the same way.

4 Krylov bases that a quantum computer can generate

4.1 The role of $f(H)$

A quantum computer cannot apply H, so the basis vectors are $v_k = f_k(H)v_0$ for functions f_k that the device can implement, typically powers $f_k = f^k$ of one unitary $f(H)$. Here $f(H)$ does *not* approximate the product Hv. It replaces the matrix–vector product as the operation that *generates the subspace*, and H itself enters only through the projected matrix $M = V^*HV$ in (4). In a Krylov method the matvec only has to supply new directions rich in the wanted eigenvector. Write $H = \sum_i \lambda_i u_i u_i^*$ with $\lambda_0 \le \lambda_1 \le \cdots$ and $v_0 = \sum_i \gamma_i u_i$. Every $f_k(H)$ has the same eigenvectors as H, so

$$v_k = f_k(H)v_0 = \sum_i f_k(\lambda_i)\gamma_i u_i,$$

and Rayleigh–Ritz with H extracts the eigenvalues from whatever space the f_k produce. This is the familiar situation of shift-and-invert Lanczos, where $(H - \sigma I)^{-1}$ builds the space but one wants eigenvalues of H, or of polynomially filtered subspace iteration. Every vector in the subspace has the form $\phi(H)v_0$ with $\phi \in \operatorname{span}\{f_0, \ldots, f_{m-1}\}$. The quality of the space is determined by how well this function space contains filters ϕ that are large at λ_0 and small on the rest of the spectrum. The choice of the f_k also fixes the structure of G and M, and therefore how many numbers the device has to estimate. The two main choices behave quite differently.

- If f_k is a polynomial of degree k, e.g. $f_k(\lambda) = T_k(\lambda/\alpha)$ (Section 4.3), then range$(V) = \mathcal{K}_m(H, v_0)$ is exactly the Lanczos space, only in a different basis.

- If $f_k(H) = U^k$ with $U = e^{-\mathrm{i}\Delta t H}$ (Section 4.2), the space consists of trigonometric polynomials in H applied to v_0. It is *not* the Lanczos space, and $U^k v_0$ is not an approximation of $H^k v_0$. The two are related only in the limit $\Delta t \to 0$. Since $U = I - \mathrm{i}\Delta t H + O(\Delta t^2)$, the scaled differences $\Delta t^{-j}(U - I)^j v_0$ tend to $(-\mathrm{i}H)^j v_0$, and $\mathcal{K}_m(U, v_0) \to \mathcal{K}_m(H, v_0)$. In this limit the vectors $U^k v_0$ become nearly identical, so G becomes singular. A practical $\Delta t \approx \pi/\|H\|_2$ is therefore a deliberate move away from the Lanczos space: one gets a different space, still containing good filters, whose basis $U^k v_0$ is far better conditioned than for small Δt (though still ill-conditioned, see Section 5).

4.2 Real-time Krylov

The most popular choice uses powers of the time evolution operator,

$$v_k = U^k v_0 = e^{-\mathrm{i}k\Delta t H} v_0, \qquad k = 0, \dots, m-1, \tag{6}$$

i.e. range$(V) = \mathcal{K}_m(U, v_0)$, a Krylov space for the unitary U rather than for H [1,4,5,7]. Because U is unitary and commutes with H,

$$G_{jk} = v_0^* U^{k-j} v_0 =: g_{k-j}, \qquad M_{jk} = v_0^* H U^{k-j} v_0 =: h_{k-j}, \tag{7}$$

with $g_{-k} = \overline{g_k}$ and $h_{-k} = \overline{h_k}$. Both G and M are *Hermitian Toeplitz*, so only the $2m$ numbers $g_0, \dots, g_{m-1}$ and $h_0, \dots, h_{m-1}$ have to be estimated. g_k is a Hadamard test with $W = U^k$. Using (1), $h_k = \sum_\ell \alpha_\ell \, v_0^* P_\ell U^k v_0$ is a weighted sum of Hadamard tests with $W = P_\ell U^k$.

Spectral picture. With the eigendecomposition from Section 4.1,

$$g_k = \sum_i |\gamma_i|^2 z_i^k, \qquad h_k = \sum_i |\gamma_i|^2 \lambda_i z_i^k, \qquad z_i = e^{-\mathrm{i}\Delta t \lambda_i}. \tag{8}$$

The data are samples of a sum of complex exponentials. This connects the method to Prony's method, matrix pencil methods and filter diagonalization in signal processing. It also shows

$$G = Z^* D Z, \qquad Z_{ik} = z_i^k, \quad D = \mathrm{diag}(|\gamma_i|^2), \tag{9}$$

so G is a weighted Gram matrix of a Vandermonde matrix with nodes on the unit circle.

Convergence. Every vector in the subspace is $p(U)v_0$ for a polynomial p of degree less than m. As for Lanczos, it follows that

$$\theta_{\min} - \lambda_0 \;\le\; (\lambda_{\max} - \lambda_0) \, \frac{1 - |\gamma_0|^2}{|\gamma_0|^2} \min_{\deg p < m} \frac{\max_{i \ge 1} |p(z_i)|^2}{|p(z_0)|^2}. \tag{10}$$

For Lanczos one has $p(\lambda)$ on the real interval $[\lambda_1, \lambda_{\max}]$ and inserts Chebyshev polynomials (Kaniel–Paige–Saad). Here p must be small on the arc of the unit circle containing $z_1, \dots, z_{n-1}$ and large at z_0. If $\Delta t(\lambda_{\max} - \lambda_0) < 2\pi$ this arc does not wrap around, and (10) gives exponential convergence in m, with a rate governed by the angular gap $\Delta t(\lambda_1 - \lambda_0)$. Rigorous versions, including noise, are given in [6, 7]. The choice of Δt is a trade-off. If it is too large, distinct eigenvalues alias to the same z_i. If it is too small, the nodes cluster and G becomes even more ill-conditioned (this is the Lanczos limit discussed in Section 4.1). A common choice is $\Delta t \approx \pi / \|H\|_2$.

The factor $|\gamma_0|^{-2}$ in (10) matters far more than classically. A random start vector has $|\gamma_0|^2 \approx 1/n = 2^{-N}$, which is useless. One needs a preparable v_0 with non-negligible overlap with the ground state u_0. Typical choices are a product state (Section 2.1) or a single standard basis vector e_b selected by a cheap classical approximation (in chemistry the Hartree–Fock state). This *state preparation* problem is shared by all quantum eigenvalue algorithms.

4.3 Chebyshev Krylov via block encodings

With a block encoding of H/α one can prepare $v_k = T_k(H/\alpha)v_0$, and then range$(V) = \mathcal{K}_m(H, v_0)$ is the *exact* Lanczos subspace [10]. Only the Chebyshev moments $\mu_k = v_0^* T_k(H/\alpha) v_0$, $k = 0, \dots, 2m-1$, are needed, because $T_j T_k = \frac{1}{2}(T_{j+k} + T_{|j-k|})$ and $xT_k = \frac{1}{2}(T_{k+1} + T_{|k-1|})$ give

$$G_{jk} = \tfrac{1}{2}\big(\mu_{j+k} + \mu_{|j-k|}\big), \qquad M_{jk} = \tfrac{\alpha}{4}\big(\mu_{j+k+1} + \mu_{|j+k-1|} + \mu_{|j-k|+1} + \mu_{||j-k|-1|}\big), \tag{11}$$

i.e. Toeplitz-plus-Hankel matrices. Convergence is that of Lanczos, but block encodings need extra qubits and deeper circuits. This variant targets future, error-corrected hardware.

4.4 Unifying view and other variants

Both variants measure *moments* of the spectral measure $d\mu(\lambda) = \sum_i |\gamma_i|^2 \delta(\lambda - \lambda_i)$ of H with respect to v_0, in the basis $e^{-\mathrm{i}k\Delta t\lambda}$ or $T_k(\lambda/\alpha)$. Solving (5) is then the familiar problem of computing Gauss-type quadrature nodes from (modified) moments [11]. Classically this is known to be ill-conditioned in the power basis and better behaved for Chebyshev moments. The quantum setting adds random noise of size ε to the moments.

Further variants include imaginary-time bases $v_k \propto e^{-k\tau H} v_0$ (QLanczos [3], where G is Hankel up to diagonal scaling), several start vectors (block or multireference Krylov [2]), and non-Krylov subspaces. Examples are $\mathrm{span}\{v_0, P_1 v_0, \ldots, P_m v_0\}$ for selected Pauli strings or similar simple matrices P_j [12], and spans of standard basis vectors e_b whose indices b are obtained by measuring a prepared state; in the latter case $G = I$ and M is computed classically [13]. The common computational core is always the pencil (5).

5 The numerical core: a noisy, ill-conditioned pencil

Ill-conditioning is intrinsic. The better the space approximates u_0, the closer the basis is to linear dependence. By (9), G inherits the exponential ill-conditioning of Vandermonde matrices with nodes clustered on an arc, weighted by the possibly tiny $|\gamma_i|^2$. Figure 1 (right) shows $\kappa_2(G)$ growing exponentially until it reaches $1/\mathbf{u} \approx 10^{16}$ for $m \approx 20$, although n is only 1024.

Noise. The device returns $\widetilde{G} = G + E_G$ and $\widetilde{M} = M + E_M$ with $\|E_G\|, \|E_M\|/\|H\| \sim \varepsilon$, and typically $\varepsilon \in [10^{-4}, 10^{-2}]$. Once $\lambda_{\min}(G) \lesssim \varepsilon$, $\widetilde{G}$ may be indefinite and the pencil $(\widetilde{M}, \widetilde{G})$ may have eigenvalues far from the spectrum of H, possibly below λ_0. The variational guarantee $\theta_{\min} \geq \lambda_0$ is lost. Solving (5) naively gives garbage (red curve in Figure 1).

Thresholding. The standard remedy, known in quantum chemistry as canonical orthogonalization, is a truncated eigenvalue decomposition. Compute $\widetilde{G} = W \Sigma W^*$, keep the eigenpairs with $\sigma_i > \tau$ (collected in W_τ, Σ_τ), and solve the $r \times r$ standard Hermitian problem

$$\left(\Sigma_\tau^{-1/2} W_\tau^* \, \widetilde{M} \, W_\tau \Sigma_\tau^{-1/2}\right) y = \theta \, y. \tag{12}$$

This is the analogue of a truncated SVD for an ill-posed problem, with τ a few times the noise level. Why does this work at all when $\kappa(G) \gg 1/\varepsilon$? Rayleigh–Ritz only needs *one* good vector in the subspace. If $u_0 \approx Vc$ with a coefficient vector c of moderate norm, the perturbation changes its Rayleigh quotient by roughly $\varepsilon\|c\|^2$, and thresholding at τ does not remove it. Epperly, Lin and Nakatsukasa [6] turn this argument into a rigorous stability theory for quantum subspace diagonalization. The error is bounded by the noiseless Krylov error plus a term controlled by ε, τ and the size of good coefficient vectors. Kirby [7] refines the analysis for the real-time basis. In practice, the achievable accuracy is set by the noise level rather than by $\kappa(G)$.

Experiment. For Figure 1 we take the Ising chain with $N = 10$, $h = 1$, the product state $v_0 = 2^{-N/2}(1, \ldots, 1)^T = a \otimes \cdots \otimes a$ with $a = (1, 1)^T/\sqrt{2}$ as start vector ($|\gamma_0|^2 \approx 0.47$) and $\Delta t = \pi/(\lambda_{\max} - \lambda_0)$. Noise is modelled as independent complex Gaussian perturbations of g_k and h_k with standard deviations ε and $\varepsilon\,(\lambda_{\max} - \lambda_0)$, respectively, with threshold $\tau = 10\varepsilon$ (median over 50 realizations). The real-time basis converges exponentially, but here more slowly than Lanczos. Even without noise, rounding limits it to about 10^{-9} because $\kappa(G) \approx 10^{16}$. With noise, the error stagnates at roughly the noise level: about $3 \cdot 10^{-6}$ for $\varepsilon = 10^{-6}$ and $6 \cdot 10^{-4}$ for $\varepsilon = 10^{-3}$. Without thresholding there is no convergence at all.

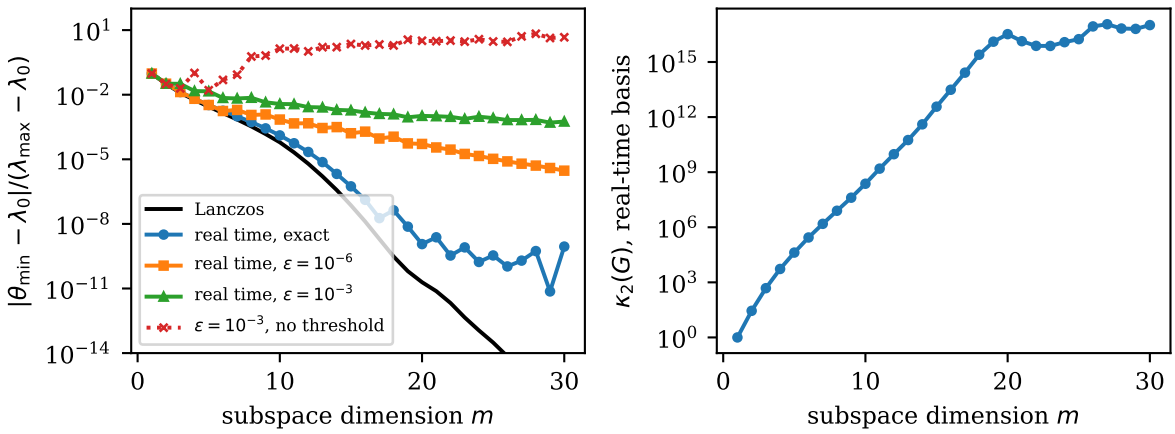


Figure 1: Ising chain, $n = 2^{10}$. Left: relative error of the smallest Ritz value for Lanczos and for the real-time Krylov method with exact and noisy matrix elements. Right: condition number of the exact Toeplitz Gram matrix G. Script: `qk_experiment.py`.

6 Cost and practical aspects

- **Shots.** The real-time method needs $O(m)$ distinct numbers. Through (1), each h_k costs up to L Hadamard tests, and each test needs $O(\varepsilon^{-2})$ shots. Grouping commuting Pauli strings and tailoring shot budgets to entries reduce this (see also Section 8.2), but sampling dominates the run time.

- **Circuit depth.** v_{m-1} requires evolution up to time $(m-1)\Delta t$. On noisy hardware the depth limits m, which favours small subspaces with a good start vector. Trotter errors mean that the implemented U commutes with H only approximately, so the Toeplitz structure of M is itself an approximation.

- **Controlled operations.** The Hadamard test needs $C(U^k)$ from (2), which is expensive. Variants avoid it when an additional vector $r \perp v_0$ with a known eigenvalue $Ur = e^{-\mathrm{i}\phi}r$ is available, e.g. from a symmetry of H; superpositions of v_0 and r then reveal $v_0^* U^k v_0$ without $C(\cdot)$.

- **Comparison with phase estimation.** Quantum phase estimation, the textbook quantum eigenvalue algorithm, computes λ_0 to accuracy ε with evolution time $O(1/\varepsilon)$ and needs error-corrected hardware (Section 2.4). Krylov methods trade circuit depth for classical post-processing and many shots. They are therefore regarded as near-term methods, and they are close relatives of recent phase estimation variants designed for the first generation of error-corrected hardware, which also process the signal g_k classically.

7 Beyond eigenvalues: matrix functions

Classically, Krylov methods serve matrix functions as much as eigenvalues. With the Lanczos basis Q_m and the tridiagonal matrix T_m,

$$v^* f(H) v \approx \|v\|^2 e_1^T f(T_m) e_1, \qquad f(H)v \approx \|v\|\, Q_m f(T_m) e_1.$$

The first is Gauss quadrature for the spectral measure, and the second is the Lanczos approximation [11]. Both carry over to the quantum setting.

The Krylov approximation in a non-orthogonal basis. With $G = R^*R$, the matrix $Q = VR^{-1}$ has orthonormal columns. Since $v_0 = Ve_1$, the Krylov (Galerkin) approximation $Qf(Q^*HQ)Q^*v_0$ can be written using only the measured matrices:

$$f(H)v_0 \approx V\, f(G^{-1}M)\, e_1, \qquad v_0^* f(H) v_0 \approx e_1^T\, G\, f(G^{-1}M)\, e_1. \tag{13}$$

Here $G^{-1}M$ is similar to the Hermitian matrix $R^{-*}MR^{-1}$, so f is evaluated at the Ritz values. In practice one works with the thresholded matrix of (12). For the Chebyshev basis, (13) is Gauss quadrature for the spectral measure μ (Section 4.4). For the real-time basis, $\mathcal{K}_m(U, v_0)$ is a Krylov space of a unitary matrix, and G is a positive definite Hermitian Toeplitz matrix, namely the moment matrix of the spectral measure mapped to the unit circle, cf. (9). The natural classical tools are then orthogonal polynomials on the unit circle: the Szegő recursion, which is the Levinson algorithm for G, and Szegő quadrature, related to the isometric Arnoldi process [32]. These orthogonalize in coefficient space while exploiting the Toeplitz structure. Their behaviour under measurement noise is a natural research question. For instance, the recursion detects loss of positive definiteness of G through reflection coefficients of modulus at least one.

Linear expansions. Alternatively, f can be expanded directly in the functions that are measured, $f(\lambda) \approx \sum_k c_k e^{-\mathrm{i}k\Delta t\lambda}$ or $f(\lambda) \approx \sum_k c_k T_k(\lambda/\alpha)$. Then $v_0^* f(H) v_0 \approx \sum_k c_k g_k$ or $\sum_k c_k \mu_k$ is *linear* in the data. Noise enters only through $\|c\|_1$, and Monte Carlo over k applies (Section 8.2). The price is that the expansion is fixed in advance for the whole spectral interval. In contrast, (13) adapts to the spectrum seen by v_0 but is nonlinear in the noisy data. Classically, this is the same trade-off as between Chebyshev (kernel polynomial) methods and Lanczos quadrature.

Which functions have been treated.

- *Time evolution* $e^{-\mathrm{i}tH}$ beyond the available circuit depth. Correlation functions are computed from a Krylov subspace built with short evolutions ("fast-forwarding") [33].
- *Resolvents* $(z - H)^{-1}$, i.e. Green's functions. Approaches include continued fractions from Lanczos coefficients obtained on a quantum computer [34], Arnoldi-type methods designed for robustness [35], and Chebyshev expansions with block encodings [36].
- *Spectral densities and step functions.* These are treated by kernel polynomial expansions [36] and via the cumulative spectral function [23].
- *Thermal quantities* with $e^{-\beta H}$. Approaches include imaginary-time methods [3] and Krylov-type methods extended to finite temperature [35].

Normalized traces $\frac{1}{n}\operatorname{trace} f(H)$, such as the density of states, are estimated with random start vectors exactly as in Hutchinson's estimator. This works on hardware [37] because the target is $O(1)$, cf. Section 8.

Vectors $f(H)v_0$. Formula (13) gives only the coefficient vector $c = f(G^{-1}M)e_1$. As for Ritz vectors, this suffices for quantities such as $x^*Ax \approx c^*(V^*AV)c$ with $x \approx f(H)v_0$. This is how correlation functions are obtained in [33]. Preparing $f(H)v_0$ itself as a vector on the device from c, by a linear combination of unitaries, succeeds with probability $\|Vc\|^2/\|c\|_1^2$, which suffers from the ill-conditioning. On error-corrected hardware one instead applies a fixed polynomial $p \approx f/\max|f|$ through a block encoding (Section 2.4). The success probability is $\|p(H/\alpha)v_0\|^2$. Up to logarithmic factors, the degree is about αt for $e^{-\mathrm{i}tH}$, about $\sqrt{\alpha\beta}$ for $e^{-\beta H}$, and about κ for H^{-1}, where $1/\kappa$ is the smallest singular value of H/α [9]. These polynomials are designed for the whole interval $[-1, 1]$, not for the spectrum seen by v_0. The adaptivity behind the superlinear convergence of classical Krylov methods is therefore not used. Whether measured moments can be used to design v_0-adapted polynomials that stay bounded on $[-1, 1]$ appears to be open.

8 Sketching and randomization

Randomized numerical linear algebra [18] offers two kinds of tools. *Sketches* compress a large dimension by a random linear map. *Monte Carlo sampling* replaces a long sum by a few randomly chosen terms. In quantum Krylov methods the first kind fails exactly where it would be most welcome, namely to repair the conditioning of G. The second kind is already central, because on a quantum device the scarce resources are shots and circuit length, not flops.

8.1 What does not work: sketching to repair the conditioning

Classically, sketched Rayleigh–Ritz [15] avoids the squaring of the condition number in Cholesky-QR (Section 3). It works with SV instead of $G = V^*V$, where $S \in \mathbb{C}^{s\times n}$ is random with $s \approx 2m$, and orthogonalizes the small matrix SV by Householder QR. Since S approximately preserves norms on range(V), one has $\kappa(SV) \approx \kappa(V)$, so nothing is squared.

On the device, an entry $s_i^* v_k$ of SV is an inner product. If $s_i = Q_i e_{0\ldots0}$ and $v_k = Q_k e_{0\ldots0}$ for unitaries with short circuits, then $s_i^* v_k = e_{0\ldots0}^*(Q_i^* Q_k)e_{0\ldots0}$ can be estimated by a Hadamard test (Section 2.5). Two obstacles appear.

- *The entries are exponentially small.* For a random unit vector s_i, $|s_i^* v_k|^2 \approx 1/n = 2^{-N}$, while the device estimates inner products only up to an additive error ε (Section 2.2). The rescaling by $\sqrt{n/s}$ that a subspace embedding needs amplifies this error by about $2^{N/2}$. This is not a weakness of one particular sketch. An *oblivious* sketch, i.e. one chosen without knowledge of V, must work for every m-dimensional subspace, and for a typical subspace the overlaps with any s fixed unit vectors are of size $n^{-1/2}$ (concentration of measure).

- *Random vectors cannot be prepared.* A random unit vector in $\mathbb{C}^n$ has no short circuit (Section 2.4).

Hadamard probe vectors, as used in diagonal estimation, remove the second obstacle but not the first. The normalized rows of the Walsh–Hadamard matrix $F = h \otimes \cdots \otimes h$ are the product states Fe_b, which take N gates. However, they form an orthonormal basis, so $\sum_b |(Fe_b)^* v|^2 = 1$ and the typical squared overlap is again $1/n$. A Hadamard sketch is just a change of basis. It helps only if the Krylov vectors are concentrated on few rows of F, and then measuring Fv samples those rows directly (Section 8.2). The subsampled randomized Hadamard transform is even designed to spread a vector evenly over all rows, which is exactly the wrong property here. Trace and diagonal estimation are different: there the target $\mathrm{trace}(A)/n$ is $O(1)$, so random or Hadamard probes work on a quantum device too. A Krylov sketch, in contrast, needs $O(1)$ overlaps with one particular subspace.

A sketch chosen *depending on* V can have $O(1)$ overlaps, e.g. $s_i = W_i v_0$ with cheap unitaries W_i. Collecting these vectors in $W = [s_1, \ldots, s_s]$, $SV = W^*V$ becomes a cross-Gram matrix, and the pencil (W^*HV, W^*V) is a Petrov–Galerkin (oblique) projection. In the ideal case, where W has orthonormal columns and range$(V) \subseteq$ range(W), one has $\|W^*Vx\| = \|Vx\|$ for all x, hence $\kappa(W^*V) = \kappa(V)$ without squaring. But preparing such a W is the orthogonalization problem again. Whether a well-conditioned, only approximately suitable W from some other cheap family helps is open.

8.2 What does work: sampling

Measurement as row sampling. Measuring a vector v draws a row index b with probability $|v_b|^2$. In randomized numerical linear algebra this is squared-row-norm sampling; for a single vector these are exactly its leverage scores. Sample-based methods use it directly [13, 19]. They measure v_0 or the Krylov vectors $U^k v_0$ many times, collect the set B of observed indices, and solve the eigenvalue problem for the principal submatrix $H(B, B)$ classically, e.g. by sparse

Lanczos. Every Pauli string is a permutation matrix times a diagonal matrix of phases, so each column of H has at most L nonzeros, which are cheap to compute from (1). The basis $\{e_b\}_{b\in B}$ is orthonormal, so $G = I$ and there is no conditioning problem. By Cauchy interlacing, $\lambda_{\min}(H(B,B)) \geq \lambda_0$ for every B, so noise in the samples only makes the choice of B less efficient and can never push the result below λ_0. In NLA terms, the quantum computer serves as a sampler for column subset selection.

The catch is that B must be manageable, so the ground state has to be concentrated on relatively few basis vectors. Which basis is right depends on the problem. For the Ising chain with $N = 10$ (script `qk_concentration.py`), the number of basis vectors carrying 99% of $\|u_0\|^2$ is 83 in the standard basis versus 467 in the Hadamard basis for $h = 0.5$, and 971 versus 16 for $h = 2$. Near the critical value $h = 1$ neither basis works well (793 versus 206 out of 1024). Under sparsity assumptions of this kind, polynomial-time convergence has been proved for the Krylov variant [19].

Monte Carlo over sums. Many quantities have the form $x = \sum_i c_i x_i$, where each $x_i \in [-1, 1]$ can only be sampled, for example as the mean of a ± 1 outcome s of a Hadamard test. Drawing i with probability $|c_i|/\|c\|_1$ and recording $\|c\|_1 \operatorname{sign}(c_i)\, s$ gives an unbiased estimator of x with variance at most $\|c\|_1^2$. The cost depends on $\|c\|_1$ and not on the number of terms. This matters when there are very many or infinitely many terms, and it is the same principle as in randomized matrix multiplication [20] and Monte Carlo trace estimation. Three uses:

- *Matrix entries.* $h_k = \sum_\ell \alpha_\ell\, v_0^* P_\ell U^k v_0$ costs $O(\alpha^2/\varepsilon^2)$ shots with $\alpha = \sum_\ell |\alpha_\ell|$, independently of L. This matches an optimal distribution of shots over the L terms, without enumerating them.

- *Time evolution (qDRIFT [21]).* Instead of a Lie–Trotter product over all L terms, one multiplies q factors $e^{-\mathrm{i}(\alpha t/q)\operatorname{sign}(\alpha_\ell) P_\ell}$, each with ℓ drawn with probability $|\alpha_\ell|/\alpha$. The circuit length depends on α and t but not on L, which pays off for Hamiltonians with very many small terms, as in chemistry. Matrix concentration inequalities show that a single random product is already accurate with high probability [22], an analysis in the spirit of randomized NLA.

- *Random times.* If the post-processing is *linear* in the signal g, one can draw a random time for every shot. Lin and Tong [23] estimate the cumulative spectral function $x \mapsto \sum_{\lambda_i \leq x} |\gamma_i|^2$ in this way; its first jump is at λ_0. Each shot needs a single Hadamard test at one random time.

Rayleigh–Ritz, in contrast, is nonlinear in the data, so in Krylov methods Monte Carlo can only be applied entry by entry.

Randomized measurements. Classical shadows [16] apply a random cheap unitary before measuring, e.g. a Kronecker product of 2×2 unitaries drawn from a fixed finite set. From the outcomes one builds an unbiased random estimate of the rank-one matrix vv^*, the *shadow*. With $O(3^w \log M/\varepsilon^2)$ shots, all M quantities v^*Pv with Pauli strings P of weight at most w (at most w non-identity factors) are estimated to accuracy ε. The logarithmic dependence on M is reminiscent of the Johnson–Lindenstrauss lemma. Off-diagonal quantities $v_j^* P v_k$ follow in the same way from the vector $\frac{1}{\sqrt{2}}[v_j; v_k]$, which uses one extra qubit as in the Hadamard test. This helps when many different numbers are needed: subspaces spanned by $P_j v_0$ for many Pauli strings [17], or many observables V^*AV. It is less useful for a single sum such as h_k, where Monte Carlo over the terms is simpler.

The classical back end. In Krylov methods the projected problems are tiny ($m \lesssim 100$), so classical sketching buys nothing there. Sample-based and shadow-based methods, however, produce large classical problems: sparse eigenproblems for $H(B, B)$ with very large $|B|$, or high-dimensional subspace problems [17]. There, randomized eigensolvers such as sketched Rayleigh–Ritz [15] apply directly.

In short, randomization is useful for saving shots and circuit length and for choosing subspaces. No sketch is known that repairs the conditioning of the Krylov Gram matrix.

9 State of the art and open questions

What is established. The framework of Sections 3–5 is well understood. Real-time and Chebyshev bases satisfy Kaniel–Paige–Saad-type convergence bounds [6, 7, 10]. For the *smallest eigenvalue*, thresholding is stable under small noise: the error is bounded by the noiseless Krylov error plus a term linear in the noise level [7], and explicit finite-shot analyses exist [24]. Hardware experiments have reached tens of qubits [8], and sampling-based hybrids come with convergence guarantees under sparsity assumptions [19]. Almost everything beyond the smallest eigenvalue under small additive noise is open. Even there, recent work disagrees on whether ill-conditioning or statistical noise is the dominant failure mode [25]. The following problems seem particularly suited to numerical linear algebra.

1. **Eigenvectors and observables.** The Ritz vector $x = Vc$ is only implicit. An observable x^*Ax requires the further matrix V^*AV, whose noise is amplified by $\|c\|^2$, and $\|c\|$ can be large in an ill-conditioned basis. Moreover, observables converge more slowly than energies. For $\|x\| = 1$ and $\theta = x^*Hx$, one has $\sin^2 \angle(x, u_0) \le (\theta - \lambda_0)/(\lambda_1 - \lambda_0)$, so an energy error η allows observable errors up to $2\|A\|_2\sqrt{\eta/(\lambda_1 - \lambda_0)}$. A $\sin\Theta$ theory in the sense of Davis and Kahan [26] for noisy, thresholded pencils seems to be missing.

2. **Excited states and spurious eigenvalues.** Thresholding has been analysed only for $\lambda_{\min}$. For higher eigenvalues there is no comparable theory, and noisy pencils produce spurious Ritz values. Recent work proposes filters that detect them [25]. Lanczos in finite precision faces a similar problem, which is handled by the Cullum–Willoughby test [27]. An analogue for noisy pencils is open.

3. **Optimal regularization and limits.** Thresholding is a heuristic. What are the optimal threshold and regularization, given the known covariance of E_G and E_M? Can the Toeplitz or Hankel structure be preserved, e.g. via structured low-rank approximation? Equally open is the converse question: what accuracy can be achieved at all from m noisy samples g_k, h_k? For real-time data this is a super-resolution problem, namely estimating the smallest frequency in (8). There, stability limits are understood through the conditioning of Vandermonde matrices. Matching them with the pencil approach is open.

4. **A posteriori error estimates.** Classical Krylov methods monitor residuals, whereas quantum Krylov methods usually report $\theta_{\min}$ without error bars. With $c^*Gc = 1$, the residual of the Ritz pair satisfies

$$\|Hx - \theta x\|^2 = c^*(V^*H^2V)\,c - \theta^2,$$

   and V^*H^2V is again Toeplitz for the real-time basis. The residual yields the classical residual bound and, given a lower bound on λ_1, Kato–Temple lower bounds on λ_0 [28]. How to estimate these reliably from noisy data, where the subtraction suffers from large $\|c\|$, is open.

5. **Orthogonal bases on the device.** Orthonormal Lanczos vectors $q_k = \pi_k(H)v_0$ can in principle be prepared. One option is to apply the Lanczos polynomials through block

encodings (cf. [10] and Section 3). Another is an operator-level Lanczos recursion, as recently proposed [29]. Both cost deeper circuits and need recurrence coefficients computed from noisy data. This is Lanczos with perturbed coefficients. A theory of the resulting loss of orthogonality, in the spirit of Paige's analysis of Lanczos in finite precision [30] with noise level ε in place of $\mathbf{u}$, would clarify when such schemes beat regularized non-orthogonal ones.

6. **Structured noise.** Most analyses assume small additive errors. Real devices add structure. Trotter errors break the commutation of U and H, and with it the Toeplitz structure of M. Decoherence damps the signal, roughly $g_k \mapsto f_k g_k$ and $h_k \mapsto f_k h_k$ with decaying f_k, i.e. an entrywise (Hadamard) product of G and M with a Toeplitz damping matrix. How such structured perturbations move Ritz values, and whether they can be undone, has received less attention than additive noise.

7. **Preparing the Ritz vector.** Many applications need the approximate eigenvector *on the device*, e.g. as start vector for phase estimation. Preparing $x = Vc = \sum_k c_k U^k v_0$ as a linear combination of unitaries (Section 2.4) succeeds with probability $\|x\|^2/\|c\|_1^2 = 1/\|c\|_1^2$. This favours coefficient vectors with small $\|c\|_1$, i.e. a Ritz problem regularized by $\|c\|_1$, which trades energy accuracy against preparation cost. We are not aware of an analysis of this trade-off.

8. **Basis design.** Non-uniform time steps, filters, block Krylov spaces with several start vectors, hybrid Krylov–sampling bases [19], and the trade-off between conditioning and aliasing in the choice of Δt.

9. **Moments, quadrature and matrix functions.** How stable is recovering Gauss or Szegő quadrature rules from noisy Chebyshev or trigonometric moments, e.g. via modified moments [11] or the Levinson–Szegő recursion [32]? When is the adaptive but nonlinear approximation (13) preferable to fixed linear expansions under noise? Can measured moments guide the choice of polynomials for block encodings (Section 7)?

10. **Experimental design.** Which entries should be measured, and with how many shots, for a given total budget?

11. **Sketching for conditioning.** Is there a sketch depending on V (Section 8) whose cross-Gram matrix W^*V avoids the squaring $\kappa(G) = \kappa(V)^2$ under additive measurement noise? Can matrix concentration inequalities give sharp shot counts for the noisy pencil?

12. **Where is the advantage?** All quantum eigenvalue methods need a start vector with non-negligible overlap $|\gamma_0|^2$. Whether such vectors exist for problems that are hard for classical methods is debated [31]. Fair comparisons with the best classical methods are therefore part of the research agenda.

For a broad survey of subspace methods on quantum computers see [14]. The analysis in [6] is written for a numerical analysis audience and is the natural next read. The list above reflects the literature as of late 2026 and is not exhaustive.

References

[1] R. M. Parrish, P. L. McMahon, Quantum filter diagonalization: quantum eigendecomposition without full quantum phase estimation, arXiv:1909.08925, 2019.

[2] N. H. Stair, R. Huang, F. A. Evangelista, A multireference quantum Krylov algorithm for strongly correlated electrons, J. Chem. Theory Comput. 16 (2020), 2236–2245.

[3] M. Motta, C. Sun, A. T. K. Tan, M. J. O'Rourke, E. Ye, A. J. Minnich, F. G. S. L. Brandão, G. K.-L. Chan, Determining eigenstates and thermal states on a quantum computer using quantum imaginary time evolution, Nature Physics 16 (2020), 205–210.

[4] K. Klymko et al., Real-time evolution for ultracompact Hamiltonian eigenstates on quantum hardware, PRX Quantum 3 (2022), 020323.

[5] C. L. Cortes, S. K. Gray, Quantum Krylov subspace algorithms for ground- and excited-state energy estimation, Phys. Rev. A 105 (2022), 022417.

[6] E. N. Epperly, L. Lin, Y. Nakatsukasa, A theory of quantum subspace diagonalization, SIAM J. Matrix Anal. Appl. 43 (2022), 1263–1290.

[7] W. Kirby, Analysis of quantum Krylov algorithms with errors, Quantum 8 (2024), 1457.

[8] N. Yoshioka et al., Krylov diagonalization of large many-body Hamiltonians on a quantum processor, Nature Communications (2025); arXiv:2407.14431.

[9] A. Gilyén, Y. Su, G. H. Low, N. Wiebe, Quantum singular value transformation and beyond, Proc. STOC 2019, 193–204.

[10] W. Kirby, M. Motta, A. Mezzacapo, Exact and efficient Lanczos method on a quantum computer, Quantum 7 (2023), 1018.

[11] G. H. Golub, G. Meurant, Matrices, Moments and Quadrature with Applications, Princeton University Press, 2010.

[12] J. R. McClean, M. E. Kimchi-Schwartz, J. Carter, W. A. de Jong, Hybrid quantum-classical hierarchy for mitigation of decoherence and determination of excited states, Phys. Rev. A 95 (2017), 042308.

[13] J. Robledo-Moreno et al., Chemistry beyond the scale of exact diagonalization on a quantum-centric supercomputer, Science Advances (2025).

[14] M. Motta et al., Subspace methods for electronic structure simulations on quantum computers, Electronic Structure 6 (2024), 013001.

[15] Y. Nakatsukasa, J. A. Tropp, Fast and accurate randomized algorithms for linear systems and eigenvalue problems, SIAM J. Matrix Anal. Appl. 45 (2024).

[16] H.-Y. Huang, R. Kueng, J. Preskill, Predicting many properties of a quantum system from very few measurements, Nature Physics 16 (2020), 1050–1057.

[17] G. Boyd, B. Koczor, Z. Cai, High-dimensional subspace expansion using classical shadows, arXiv:2406.11533, 2024.

[18] P.-G. Martinsson, J. A. Tropp, Randomized numerical linear algebra: foundations and algorithms, Acta Numerica 29 (2020), 403–572.

[19] Yu, Robledo Moreno, Iosue, et al., Quantum-centric algorithm for sample-based Krylov diagonalization, arXiv:2501.09702, 2025.

[20] P. Drineas, R. Kannan, M. W. Mahoney, Fast Monte Carlo algorithms for matrices I: approximating matrix multiplication, SIAM J. Comput. 36 (2006), 132–157.

[21] E. Campbell, Random compiler for fast Hamiltonian simulation, Phys. Rev. Lett. 123 (2019), 070503.

[22] C.-F. Chen, H.-Y. Huang, R. Kueng, J. A. Tropp, Concentration for random product formulas, PRX Quantum 2 (2021), 040305.

[23] L. Lin, Y. Tong, Heisenberg-limited ground-state energy estimation for early fault-tolerant quantum computers, PRX Quantum 3 (2022), 010318.

[24] G. Lee, D. Lee, J. Huh, Sampling error analysis in quantum Krylov subspace diagonalization, Quantum 8 (2024), 1477.

[25] Oliveira, Ziems, Glaser, preprint arXiv:2604.11532, 2026.

[26] C. Davis, W. M. Kahan, The rotation of eigenvectors by a perturbation. III, SIAM J. Numer. Anal. 7 (1970), 1–46.

[27] J. K. Cullum, R. A. Willoughby, Lanczos Algorithms for Large Symmetric Eigenvalue Computations, Vol. I, Birkhäuser, 1985.

[28] B. N. Parlett, The Symmetric Eigenvalue Problem, SIAM, 1998.

[29] Orthogonal quantum Krylov diagonalisation, preprint arXiv:2607.09476, 2026.

[30] C. C. Paige, Accuracy and effectiveness of the Lanczos algorithm for the symmetric eigenproblem, Linear Algebra Appl. 34 (1980), 235–258.

[31] S. Lee et al., Evaluating the evidence for exponential quantum advantage in ground-state quantum chemistry, Nature Communications 14 (2023), 1952.

[32] W. B. Gragg, Positive definite Toeplitz matrices, the Arnoldi process for isometric operators, and Gaussian quadrature on the unit circle, J. Comput. Appl. Math. 46 (1993), 183–198.

[33] C. L. Cortes, A. E. DePrince III, S. K. Gray, Fast-forwarding quantum simulation with real-time quantum Krylov subspace algorithms, Phys. Rev. A 106 (2022), 042409.

[34] T. E. Baker, Lanczos recursion on a quantum computer for the Green's function and ground state, Phys. Rev. A 103 (2021), 032404.

[35] Nelson, Baczewski, Estimating Green's functions with a robust quantum Arnoldi method, arXiv:2605.22920, 2026.

[36] P. Rall, Quantum algorithms for estimating physical quantities using block encodings, arXiv:2004.06832, 2020.

[37] Summer et al., Calculating the many-body density of states on a digital quantum computer, arXiv:2303.13476, 2023.